Repair ledger — ODIN Claim Reasoner

Issues found and fixed during the build, in Project Witness's "make claims, attack claims, repair claims, narrow claims" spirit. Newest first.

R1 — FIRST BUILD (v0.1.0): ODIN Claim Reasoner

First ODIN build — Move 10 of Project Witness, and the first piece of ODIN. It accepts already-structured claims, validates them deterministically, detects direct structural contradictions, preserves support references, and emits a canonical reasoning report and proof trace plus a strict checksums file. It is not connected to the workflow yet.

Scope held deliberately narrow: structured claims only; direct contradiction detection only; no natural language, no LLM, no ontology expansion, no

probabilistic reasoning, no action gating, no signing, no anchoring. ODIN proves
structural consistency only — it never proves a claim TRUE.

Design decisions worth recording:

- **Honest, structural status vocabulary.** Reasoning statuses are `CONSISTENT` / `CONTRADICTION_DETECTED` / `INVALID_INPUT`; claim statuses `ACCEPTED` / `CONTRADICTED` / `INVALID`. No status asserts truth/falsity. A regression test (`TestHonesty`) asserts no status contains `TRUE` / `FALSE` / `PROVEN`, the `known_limitations` disclaim proving truth, and the README says so.
- **Object type fidelity in contradiction detection.** Objects are compared by `(type_name, value)`, so JSON `true` and `1` (and `false` and `0`) are treated as DIFFERENT objects rather than collapsing via Python's `True == 1`. Without this, a boolean claim and an integer claim could be silently mis-grouped. Covered by `test_bool_and_int_not_equal_objects` and a selfcheck gate.
- **Order-independent, deterministic detection.** Claims are grouped by (subject,

predicate) in first-appearance order; each unordered pair is checked once; different-object takes precedence over opposite-polarity. Detection is identical regardless of input order (`test_contradiction_detection_order_independent`).

- **Verify recomputes, it does not trust the report.** `verify` independently recomputes the contradiction set from the canonical claims and requires the report's `contradiction_count` and `reasoning_status` to match — so a tampered report that lies about its own status is caught (`CONTRADICTION_COUNT_MISMATCH` / `STATUS_MISMATCH`), even after the checksums are resynced.
- **Strict schema + checksums.** Exact field sets for the claim set, each claim, and each support item; duplicate claim/support IDs refused; floats refused; empty support refused. Checksums file: exactly three keys, no duplicates/unknowns, valid lowercase digests only. Unknown extra files in the output are refused.
- **Aletheia import is integrity-only.** `import-aletheia` requires `evidence_status == VERIFIED_PRESENT` and emits only `evidence.present` and

`evidence.integrity_verified` claims — never a claim about the truth of contents (`test_29`, `test_30`). It reuses the real Aletheia v0.1 manifest shape.

- **Reused the stack's honest patterns:** deterministic canonical JSON; atomic writes (`os.replace`); refuse a non-empty output directory; `require()` not bare assert; in-process selfcheck that never launches the unit suite.

Release gate (Coordinator v0.1.8 / Companion v0.1 / Aletheia v0.1 lesson applied directly): no subprocess orchestrator in the freeze gate. The authoritative gate is the explicit command list in RELEASE_COMMANDS.md plus the static `verify_release_manifest.py` (launches no subprocesses; additionally checks the README scopes ODIN to structured claims only and disclaims proving truth). The manual smoke test is documented but not part of the gate.

Verified: all release-gate commands pass individually from a clean extraction (see TEST_RESULTS.md); selfcheck 8/8 in-process gates; verify_release_manifest.py passes (8/8, no subprocesses); the manual smoke test produces

`reasoning_status = CONSISTENT` and the ODIN output verifies; both contradiction rules fire correctly; tamper to any output file, the checksums, the file set, the report's claimed counts, or the proof trace's claim references is detected with the correct reason; and the optional Aletheia import (run against a real Aletheia v0.1 manifest) yields only integrity/presence claims that then reason as CONSISTENT.